Topic: Hand Gesture Recognition

Aiya Zhakupova SE-2409

INTRODUCTION

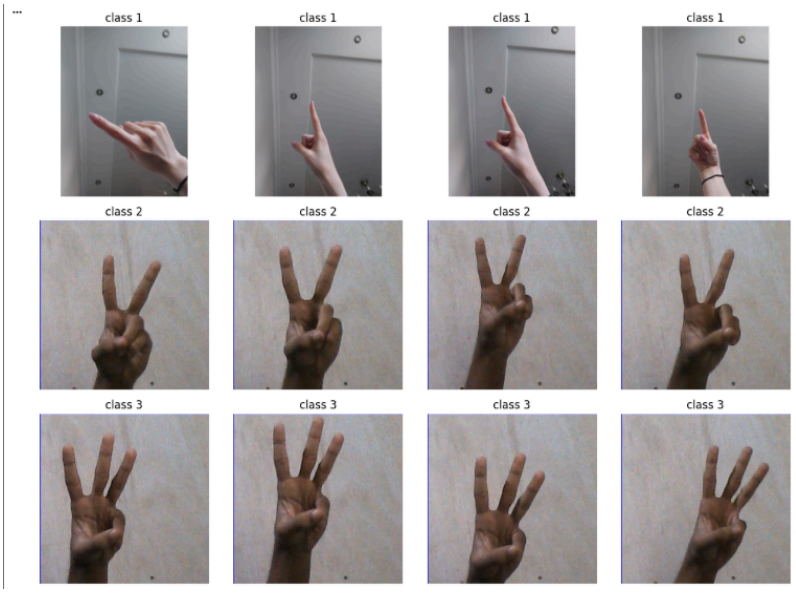
Hand gesture recognition is a computer vision task that aims to automatically identify the meaning of a hand gesture from an image. It is a useful application of image classification because it can be used in areas such as human-computer interaction, accessibility systems, education, and touchless interfaces. However, recognizing hand gestures from photographs can be challenging because the appearance of a hand may change depending on its position, orientation, lighting conditions, background, and distance from the camera.

DATASET

The dataset used in this study is a combination of images obtained from a publicly available Kaggle dataset and photographs collected by me. The Kaggle images were used as additional samples for the three target gesture classes, while the collected photographs were used to provide original examples of the same gestures.The images are organized in a standard “one folder per class” layout, where the folder name serves as the class label.

Class	Meaning	Number of images
1	Hand showing one finger	100
2	Hand showing two fingers	100
3	Hand showing three fingers	100

Table 1. Dataset composition



Picture 1. Example images from dataset

PREPROCESSING

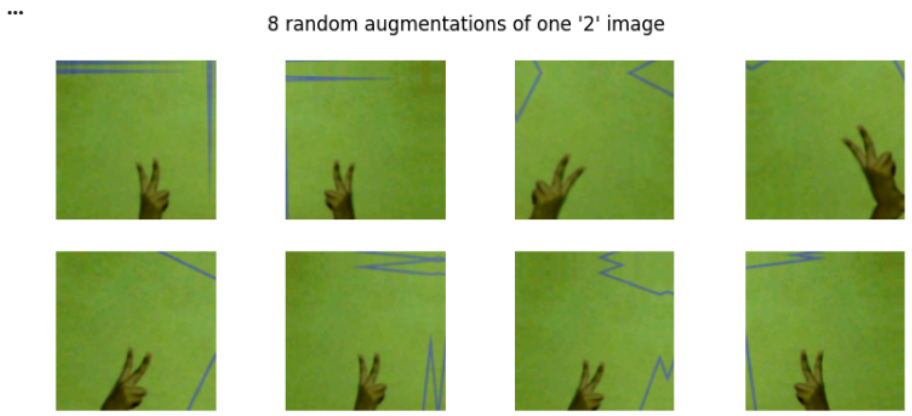
Resizing. Phone photographs have different, large resolutions, but a neural network needs a fixed input shape because the same weight matrices are applied to every image. All images are therefore resized to 128x128 pixels with three colour channels. This size is small enough to train quickly on a small dataset, yet large enough for finger shapes to remain clearly visible.

Normalization. Raw pixel intensities are integers in {0,...,255}. Each value is divided by 255 so that inputs lie in [0,1]. Small, bounded inputs lead to more stable gradients and faster, more reliable convergence than raw 0-255 values. For the baseline model, each normalized image is additionally flattened into a single vector of 128*128*3 = 49152 numbers, since Logistic Regression operates on flat feature vectors rather than on two-dimensional grids.

DATA AUGMENTATION

With about 70% of a few hundred images available for training, a flexible model can simply memorize the training photos instead of learning the general concept of each gesture; this failure mode is called overfitting. To reduce it, the training images are randomly transformed on the fly, differently in every epoch, using label-preserving transformations: random

horizontal flip, random rotation of up to about +/-29 degrees, random zoom of up to -+15%, random contrast change of up to +/-15%, imitating different lighting; random translation of up to +/-10%, imitating an off-centre hand. Augmentation is applied only to the training data; the validation and test images are left untouched because they must simulate real, unseen photographs. Since the model never sees exactly the same pixels twice, it is pushed to learn features that are robust to position and lighting rather than exact pixel patterns.



Picture 2. Data augmentation results

Baseline model: Logistic Regression

The baseline is multinomial Logistic Regression trained on the flattened, normalized pixel vectors (scikit-learn [3], max iter=2000). It learns a linear decision boundary in a 49,152-dimensional space. Crucially, it has no notion of spatial structure: pixel (0,0) and pixel (64,64) are simply two unrelated numbers, and a finger that shifts a few pixels looks like a completely different input. This makes it a deliberately weak baseline whose purpose is to set a lower bound that any more sophisticated model must beat to justify its added complexity.

Improved model: A small CNN trained from scratch

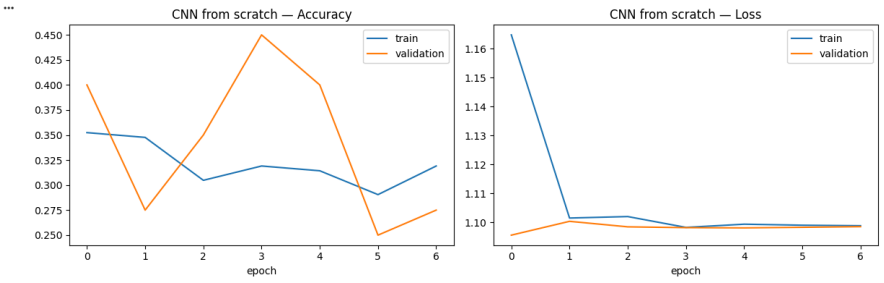
A CNN replaces the fully independent treatment of pixels with convolutional layers. Each layer slides small learnable filters (3x3 windows) over the image; a filter learns to respond to a local pattern such as an edge or a finger outline, and the same filter is applied at every

position, so a pattern is recognised wherever it appears. Max-pooling layers then halve the spatial resolution while keeping the strongest responses, which reduces computation and adds tolerance to small shifts.

CONCLUSION

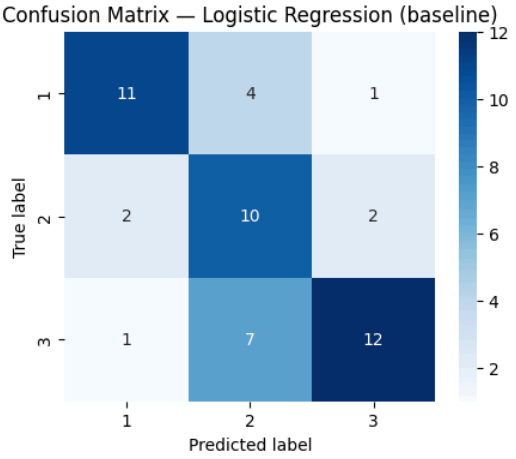
This project compared Logistic Regression and a small CNN for classifying three hand gestures: one, two, and three fingers. Both models were evaluated on the same 45-image test set using accuracy, precision, recall, F1-score, and confusion matrices.

Logistic Regression achieved 66.0% accuracy and an F1-score of 0.663, while the CNN achieved 36.0% accuracy and an F1-score of 0.176. Therefore, Logistic Regression performed better on this dataset. Although CNNs can learn spatial features from images, this CNN was trained from scratch using only 210 training images and had approximately 1.07 million trainable parameters. The training accuracy remained around 30–35%, suggesting that the model did not learn the gesture classes effectively.



Picture 3. CNN's learning process

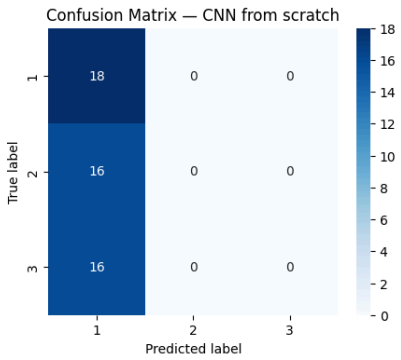
	precision	recall	f1-score	support
1	0.79	0.69	0.73	16
2	0.48	0.71	0.57	14
3	0.80	0.60	0.69	20
accuracy			0.66	50
macro avg	0.69	0.67	0.66	50
weighted avg	0.70	0.66	0.67	50



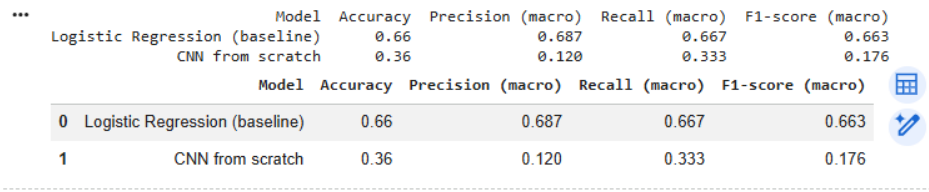
Picture 4. Confusion matrix of Logistic Regression

	precision	recall	f1-score	support
1	0.36	1.00	0.53	18
2	0.00	0.00	0.00	16
3	0.00	0.00	0.00	16
accuracy			0.36	50
macro avg	0.12	0.33	0.18	50
weighted avg	0.13	0.36	0.19	50

```
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classificat
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(resu
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classificat
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(resu
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classificat
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(resu
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classificat
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(resu
```



Picture 5. Confusion matrix of CNN



	Model	Accuracy	Precision (macro)	Recall (macro)	F1-score (macro)
0	Logistic Regression (baseline)	0.66	0.687	0.667	0.663
1	CNN from scratch	0.36	0.120	0.333	0.176

Picture 6. Results of comparison

REFERENCES

TensorFlow. TensorFlow API Documentation. Available at: https://www.tensorflow.org/api_docs

Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., and Fei-Fei, L. (2009). ImageNet: A Large-Scale Hierarchical Image Database. CVPR 2009.

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12.

A. Dalvi, "Hand Gestures Dataset," Kaggle, 2020. Available: https://www.kaggle.com/datasets/aadeshdalvi/hand-gestures-dataset